FROM CURVE TO AUTONOMY

Clothoid-R

Chungbuk national univ.

Seminar

충북대학교 지능로봇공학과 구자경

2026.03.30

AVLAB AUTONOMOUS VEHICLE CBNU

세미나 일정

기초 ROS / Gazebo

Control / GPS

LiDAR

Camera / Calibration

Planning

통합 자율주행

SUN	MON	TUE	WED	THU	FRI	SAT
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
	기초 ROS / Gazebo				Control / GPS	
29	30	31				
	LiDAR				Camera / Calibration	

세미나 교육을 듣고 본인이 제일 재밌어 보이고 하고 싶은 파트 선택

목차

인지팀 소개

인지팀 역할
- 인지팀 담당 및 요구사항
- 인지팀 포지션 별 역할

라이다 소개
- LiDAR
- 사용 라이다 소개 및 역할
- 포인트 클라우드
- Challenge points

코드 구성
- 인지 파이프라인
- 라이다 파이프라인
- 파라미터

실습
- LiDAR Sensor Data 확인
- 코드 실행 및 구조 확인
- 코드 및 파라미터 수정 실습

인지팀 소개

구자경

Interest: Sensor Fusion, DL

허주영

Interest: Camera-LiDAR Calibration

김민재

Interest: model compression, DL

지연수

Interest: Camera, Gen AI, DL

유인석

Interest: Perception, H/W

인지팀 담당

동적환경에서 카메라, 라이다를 이용한 강인한 객체 인지

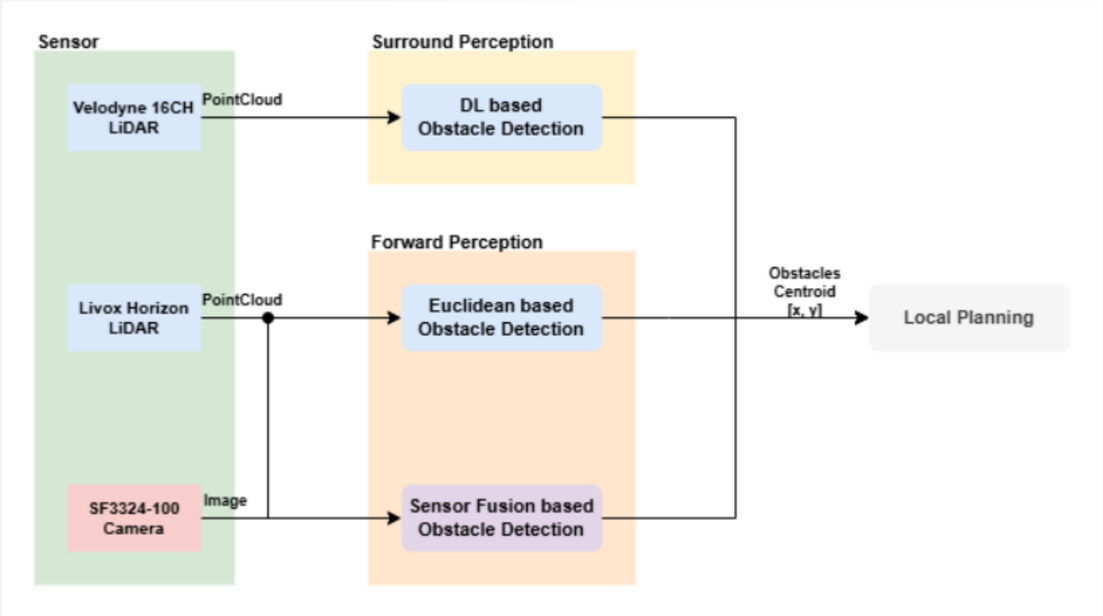
Velodyne VLP-16

Livox Horizon

SF3324-100

요구사항

10M 이내 차량 및 장애물 탐지 → 객체 중심 X, Y 좌표 Publish



인지팀 포지션 별 역할

인지팀(Perception) 팀

자율주행 시스템의 "눈" 역할. 센서 데이터로부터 차량 주변 환경을 인식하고, 후단 모듈에 전달할 정보를 생성하는 팀

영상인식 (Vision Perception)

이런 일을 해요

- 카메라 영상 기반 객체 검출 및 분류 (차량, 드럼, 라바콘 등)
- 2D 이미지 3D 공간 변환을 통한 입체 인식
- 딥러닝 모델 경량화 및 임베디드 보드 탑재 최적화

여기서 이런 것을 얻어갈 수 있어요

- Python 기반 딥러닝 모델 설계·학습·평가 전 과정
- 객체 검출, 세그멘테이션 등 컴퓨터비전 핵심 태스크 실전 경험
- 모델 경량화 및 임베디드 환경 최적화 기술
- 논문 읽고 직접 구현해보는 연구 습관

이런 분이면 좋아요

- Python, C++를 다뤄보신 분
- 디지털 영상처리, 딥러닝 관련 수업을 들어보신 분
- Computer Vision과 DeepLearing에 관심 있으신 분

라이다 인식 (LiDAR Perception)

이런 일을 해요

- LiDAR 3D 포인트 클라우드 기반 객체 탐지 및 분류
- 바닥면 분리, 포인트 클러스터링, 도로 영역 추출
- 딥러닝 기반 3D 객체 탐지 모델 학습 및 성능 개선

여기서 이런 것을 얻어갈 수 있어요

- 3D 기하학, 좌표 변환, 선형대수의 실전 응용
- ROS 환경에서 LiDAR 데이터 직접 수집·처리 경험
- 3D 인식 분야 최신 논문 구현 경험
- GPU 가속을 활용한 포인트 클라우드 대량 처리 경험

이런 분이면 좋아요

- Python, C++를 다뤄보신 분
- 선형대수학, 로봇프로그래밍, 센서 및 딥러닝 관련 수업을 들어보신 분
- 3D LiDAR와 DeepLearing에 관심 있으신 분

인지팀 포지션 별 역할

인지팀(Perception) 팀

자율주행 시스템의 "눈" 역할. 센서 데이터로부터 차량 주변 환경을 인식하고, 후단 모듈에 전달할 정보를 생성하는 팀

센서퓨전 (Sensor Fusion)

이런 일을 해요

- Camera, LiDAR 인식 결과 통합 알고리즘 개발
- 다중 객체 동시 추적(Multi-Object Tracking) 로직 설계
- 센서 간 시간 동기화 및 좌표 정합

여기서 이런 것을 얻어갈 수 있어요

- 칼만 필터 등 확률 기반 추정 알고리즘 설계·구현 능력
- 확률/통계 이론의 실전 적용 경험
- 각 센서의 특성과 한계에 대한 이해
- 수학적 모델이 실제 데이터에서 동작하는 걸 체감하는 경험

이런 분이면 좋아요

- Python, C++를 다뤄보신 분
- 확률/통계, 선형대수학, 로봇프로그래밍 수업을 들어보신 분
- 여러 센서와 E2E, Deep Learning에 관심 있으신 분

파이프라인 최적화 (Optimization)

이런 일을 해요

- 인지 모듈 간 파이프라이닝 설계 및 자원 스케줄링
- CPU/GPU 자원 배분, 멀티스레딩 구조 설계
- 파이프라인 전체 지연 시간 측정 및 병목 분석·해결

여기서 이런 것을 얻어갈 수 있어요

- OS에서 배운 개념(스레드, 스케줄링, 메모리 관리)의 실전 적용
- C++ 멀티스레딩 프로그래밍 경험
- CPU/GPU 프로파일링과 병목 분석 능력
- ROS 내부 통신 구조와 최적화 기법에 대한 이해

이런 분이면 좋아요

- C++를 다뤄보신 분
- 로봇프로그래밍, 운영체제 수업을 들어보신 분
- 기존 코드 분석, 개선 경험이 있으신 분
- 임베디드 SW 엔지니어 직무에 관심 있는 분

인지팀 포지션 별 역할

인지팀(Perception) 팀

자율주행 시스템의 "눈" 역할. 센서 데이터로부터 차량 주변 환경을 인식하고, 후단 모듈에 전달할 정보를 생성하는 팀

포지션 별 공통 사항

이렇게 진행돼요

현재 코드 분석 후 개선안 탐색 → 개선 코드 개발 → 담당 분야 테스트 및 평가
→ 인지 팀 통합 테스트 및 평가 → 전체 통합 테스트 및 평가

여기서 이런 것을 얻어갈 수 있어요

- Python + C++ 개발 및 적용 경험
- ROS 센서 파이프라인 구축 경험
- Linux 개발 환경 + Git 협업 경험
- 대량 센서 데이터 처리 경험
- 논문 → 구현 → 검증 연구 사이클 경험
- 팀원 간 커뮤니케이션 경험

나에게 맞는 포지션

이런 게 끌린다면		여기가 맞을 수 있어요
딥러닝, 이미지, 모델 학습	→	영상인식
3D 데이터, C++, 기하학	→	라이다 인식
확률/통계, 센서, 딥러닝	→	센서 퓨전
OS, 멀티스레딩, 성능 튜닝	→	파이프라인 최적화

라이다 소개

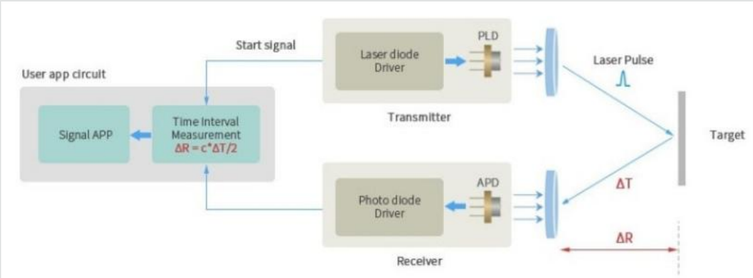
LiDAR Light Detection And Ranging

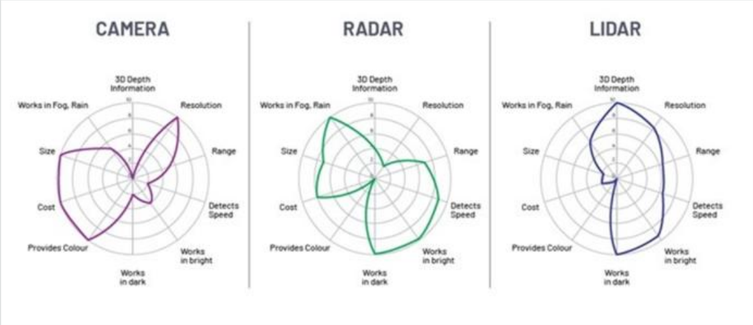
레이저를 이용하여 주변 환경의 거리정보를 측정하는 센서

측정 원리

레이저 펄스 발사 - 물체에 반사 - ToF(Time of Flight) 측정 - 거리 계산

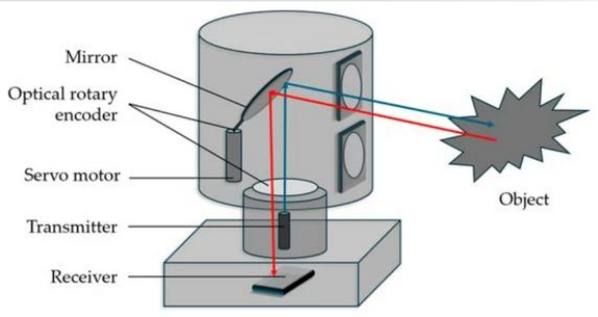
Distance = (c * Δt) / 2





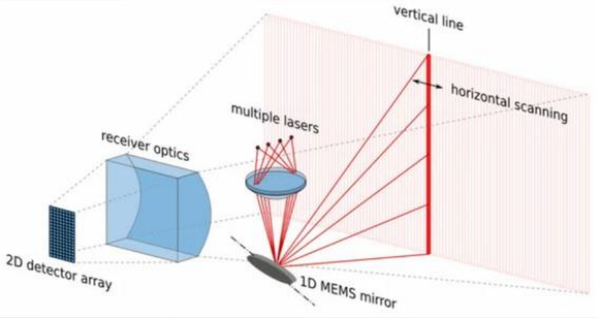
분류

Mechanical Rotating LiDAR



- 센서 전체가 회전하며 레이저 스캔
- 일반적으로 360도 수평 시야(FOV) 제공
- 모터 및 회전 구조 포함
- 내구성 및 진동/충격에 취약할 수 있음
- 주요 제조사: Velodyne, Ouster, Hesai

Solid-state LiDAR(MEMS, OPA, Flash)



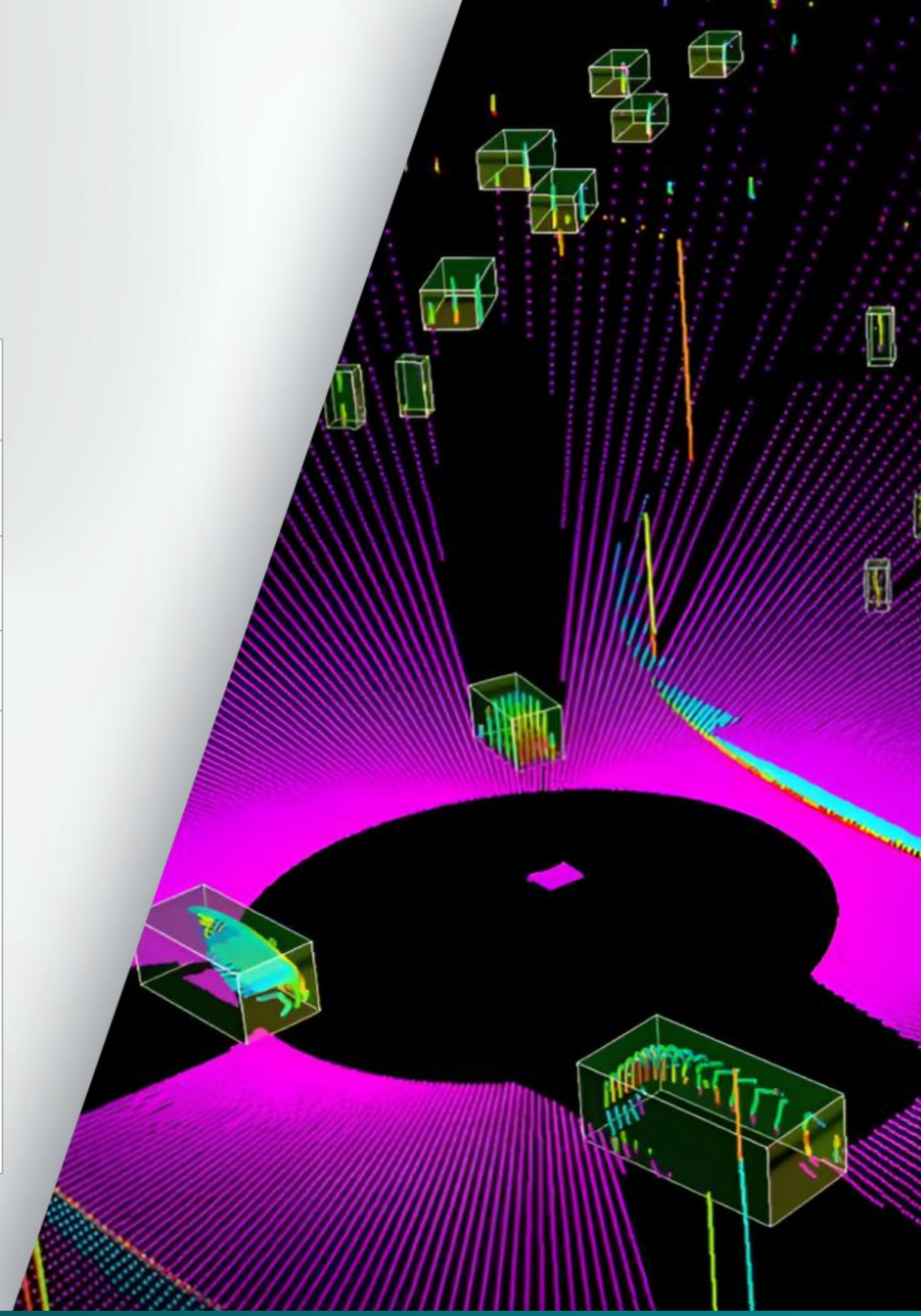
- 내부 스캔 구조 (MEMS / OPA / Flash) 사용
- 소형화 및 차량 장착에 유리
- 기계적 움직임이 적어 내구성 및 신뢰성 높음
- 일반적으로 수평 시야(FOV)가 제한적
- 주요 제조사: Livox, HL Klemove, LG Innotek, Luminar

* 참고 자료: https://www.mdpi.com/1424-8220/26/1/1

라이다 소개

사용 라이다 소개 및 역할

센서명	Velodyne VLP-16	Livox Horizon
제조사	Velodyne LiDAR	Livox
FOV(horizonal, vertical)	360° × 30°	81.7° × 25.1°
역할	차량 주변 전방위 환경 인지	차량 전방 고해상도 인지
특징	- 16채널 기계식 회전 LiDAR - 360도 주변 환경 스캔 가능 - 최대 약 100m 탐지 - 최대 약 600,000 pts/s 생성	- 비반복 스캔 방식 LiDAR - 높은 포인트 밀도 제공 - 최대 약 260m 탐지 - 최대 400,000 pts/s (dual return)



포인트 클라우드

Point Cloud

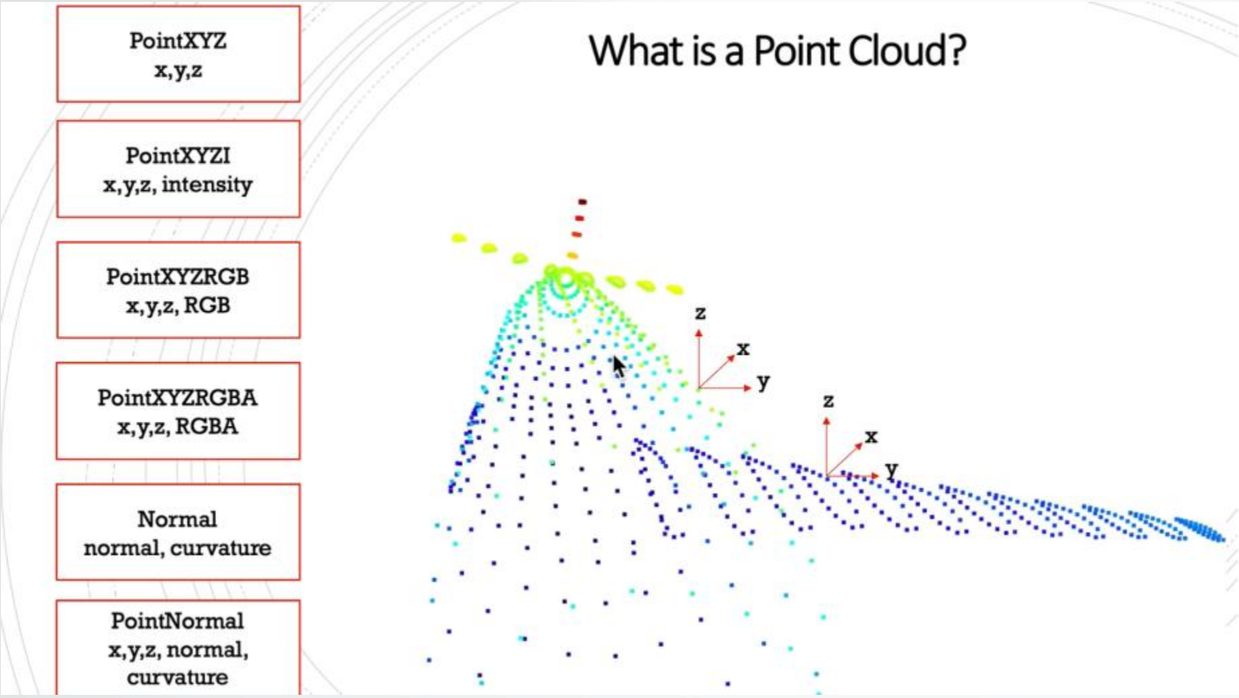
LiDAR 센서가 생성하는 3차원 공간상 점들의 집합 데이터

데이터 형태

x, y, z, intensity, (ring, timestamp)

특징

1. 비정형 데이터 - 구조화된 틀 없이 점들의 집합 형태로 표현
2. 거리 기반 밀도 변화 - 센서로부터 멀어질 수록 밀도가 작아짐
3. 노이즈 존재 - 환경으로 인한 오차, 센서 자체 오차, 센서 부착 위치에 따른 오차 등



LiDAR를 이용한 인지의 최종 목표 ➡ 점들의 집합(Point Cloud)을 의미 있는 물체 단위로 해석하는 것

Challenge Points

Sparse & Non-Uniform Sampling
LiDAR 포인트의 밀도는 거리 증가에 따라 감소
인지 범위내 밀도가 불균형

Limited Semantic Information
LiDAR는 geometry 정보만 제공
거리, 3D 구조, 반사강도는 알 수 있지만 의미 정보 제한으로 객체 분류는 어려움

Environmental Sensitivity
환경 조건에 따라 센서 성능이 떨어질 수 있음
비 / 안개 / 눈 등에 취약함

Segmentation Ambiguity
Point Cloud는 객체 경계가 모호하기 때문에 객체 분리가 어렵고 객체 형태가 불완전함

Ground Dominance
객체 포인트 대비 지면 포인트 비율이 매우 높고 클러스터링 시 객체와 지면이 혼합 될 수 있음

Real-time Processing Requirements
자율주행에서 사용할 때는 실시간 처리가 필수이며, 평균적으로 10Hz로 작동함
즉, 한 프레임을 100ms 이내에 처리하여 정확하게 객체를 탐지해야함

Sensor Fusion

multi-step processing
(Pipeline)

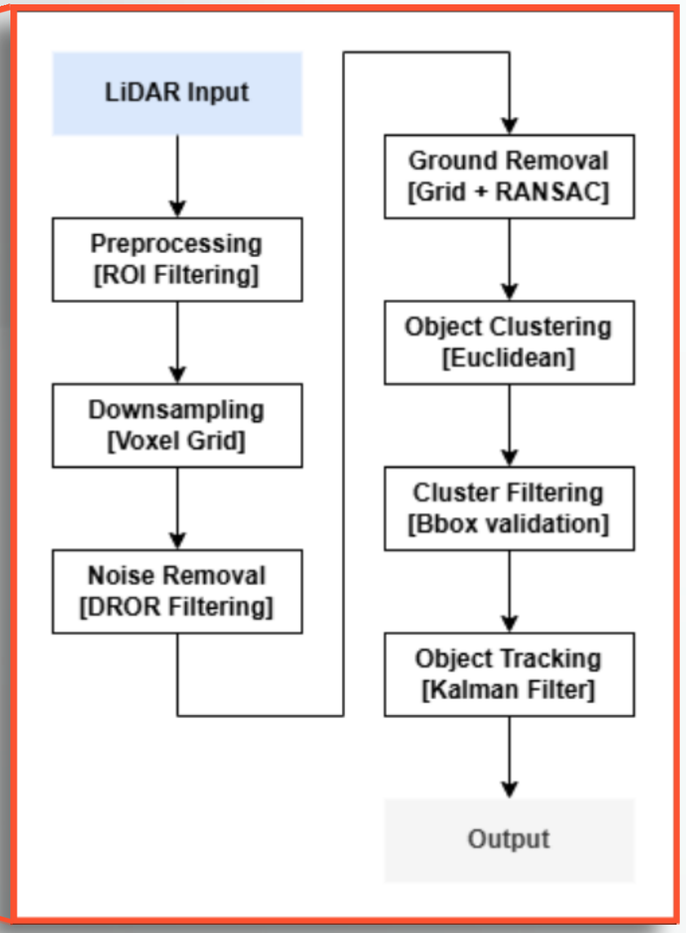
코드 구성

라이다 파이프라인

lidar_clustering_simple

Description:
object detection node based on euclidean clustering

function:
- Pitch calib
- ROI
- Voxel DROR
- Ground Remove(RANSAC)
- Euclidran Clustering



코드 구성

라이다 파이프라인

ROI Filtering / Voxel Grid

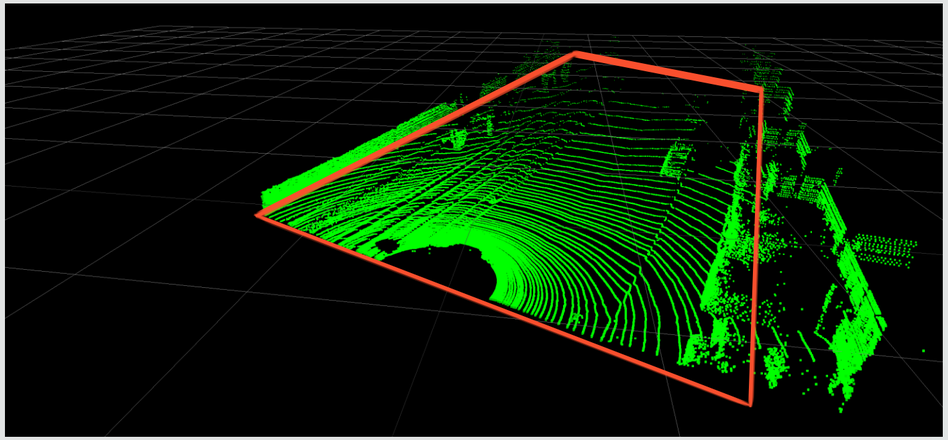
➡ 연산량 감소 및 노이즈 제거 목적

ROI Filtering

공간상의 관심영역을 설정하여 관심 영역 외의 Points 일괄 제거

- 관심 영역 외에 있는 불필요한 포인트 제거
- 범위 밖 포인트를 초반에 잘라내서 이후 모든 연산 부담을 줄임
- O(n)으로 이후 파이프라인에서 연산의 입력을 줄여주는 역할

ex)

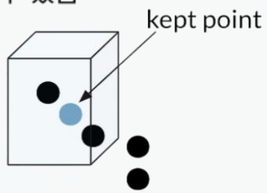


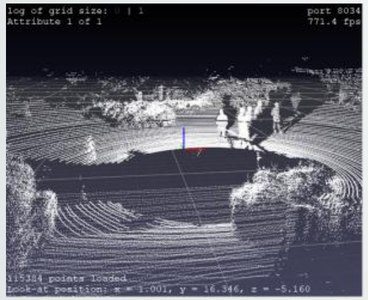
* ROI: Region of Interest, 관심 영역

Voxel Grid

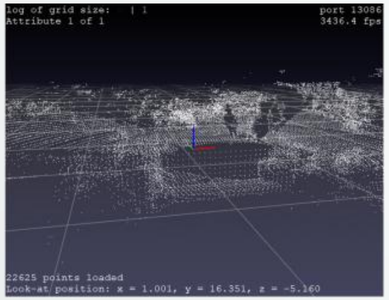
전역 Points 밀도를 일정하게 만들어 파라미터 튜닝 난이도 완화

- 공간을 일정 크기의 Voxel Grid로 나누고 3차원 Voxel 내에서 대표점 하나만 놔두고 다른 점들을 제거
- 의미있는 물체의 형상은 유지시키며 포인트들의 밀도만 낮춰 후속단계 연산량 감소와 Outlier 제거
- Voxel Size가 너무 크면 소형 객체 소실 위험이 있고 너무 작으면 효과가 없을 수 있음

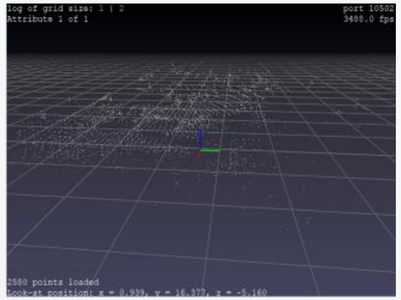




Lidar raw data (115,384 points)



Lidar raw data (22,625 points)
Voxel Size = 0.2



Lidar raw data (2,580 points)
Voxel Size = 1

*Voxel: 부피(Volume)와 픽셀(Pixel)의 합성어로, 3차원 공간에서 정육면체 형태의 격자 단위 값

코드 구성

라이다 파이프라인

DROR Filter / Grid & RANSAC

➡ 외란(Outlier) 제거 및 지면 포인트 제거

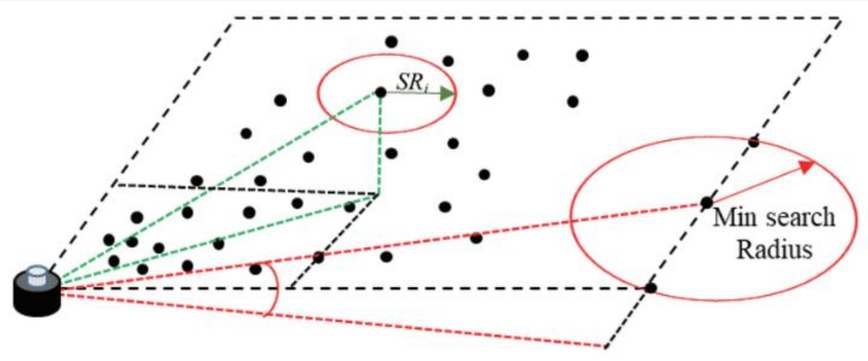
DROR(Dynamic Radius Outlier Removal) Filter

거리 기반 외란 제거

LiDAR는 거리 증가에 따라 포인트 밀도가 감소

고정 반경 필터 대신 거리 기반 반경을 적용

주변 환경에 의한 노이즈와 객체 후보 Points들과 관련 없는 저밀도 Points 제거



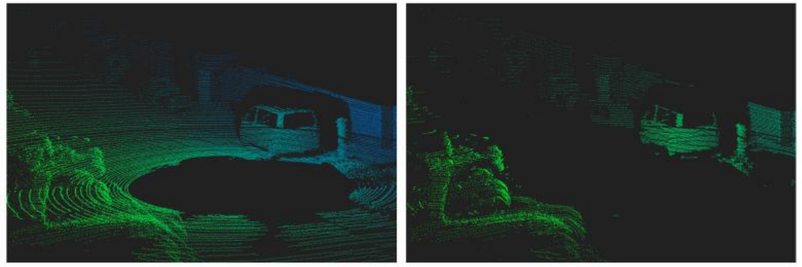
Grid & RANSAC

지면 제거

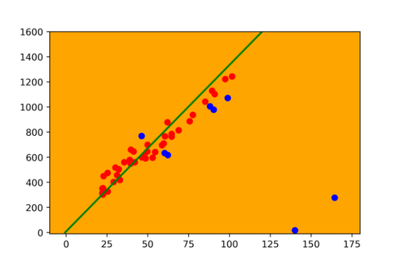
공간을 2차원 Grid cell로 나누어 각 cell의 최저 높이를 기준으로 일정 높이 이내의 점들을 제거

RANSAC을 이용하여 평면 모델 탐색 후 전역 평면과 경사면 제거

지면 제거를 통해 불필요한 연산량을 줄이고 최종 객체 탐지 정확도 향상



(a) Raw LiDAR data. (b) Filtered environment and objects data.



* RANSAC(Random Sample Consensus): 데이터셋에 이상치가 많을 때, 무작위 샘플링을 반복하여 가장 합의가 잘 되는 모델을 찾아내는 추정 알고리즘

코드 구성

라이다 파이프라인

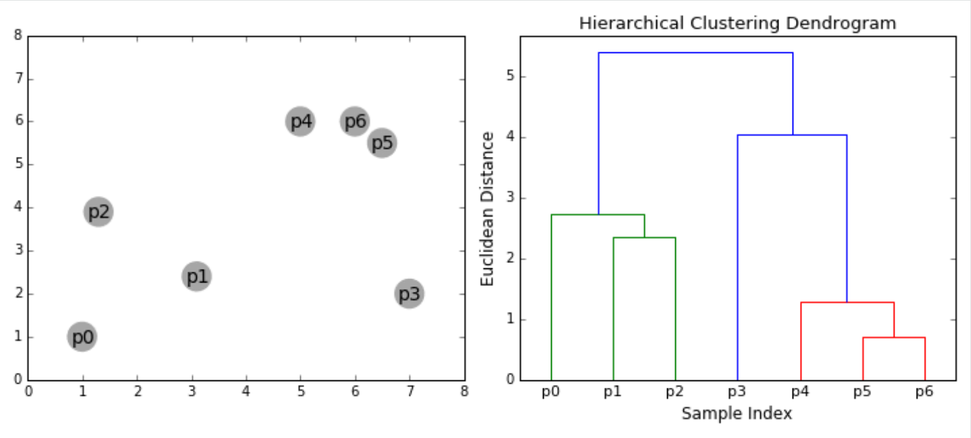
Euclidean Clustering

Euclidean Clustering

Point Cloud에서 공간적으로 가까운 포인트들을 Euclidean distance를 기반으로 군집화

거리가 멀어질수록 Point 간격이 희박 ➡ R = base_radius + scale * |distance|

1. Point 하나를 선택
2. KD-Tree로 반경 R 이내의 이웃 탐색
3. 이웃이 5개 이상이면 클러스터 ID 부여
4. 이웃들도 BFS로 재귀 탐색



Cluster Merge

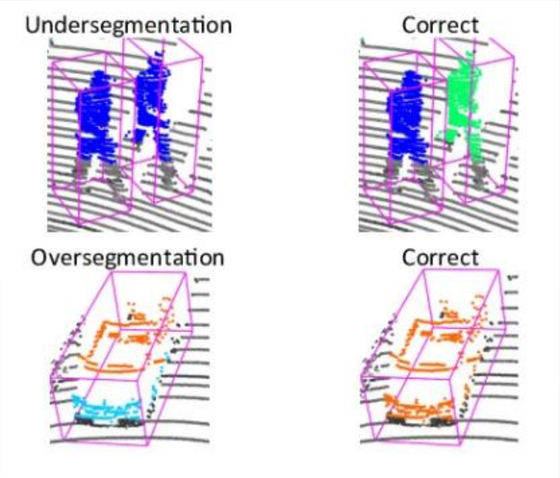
임계값 이하 거리의 클러스터 병합

여러 이유로 하나의 객체가 여러 클러스터로 분리되는 문제(Oversegmentation)

➡ 노이즈, 모션블러, 포인트 밀도, 객체 형상 등

각 클러스터 중심점 간 거리 계산 후, 임계값 이하 클러스터 병합

동일 객체 분리 문제 완화, Tracking 성능 향상



코드 구성

라이다 파이프라인

Bbox Validation / Kalman Filter Tracking

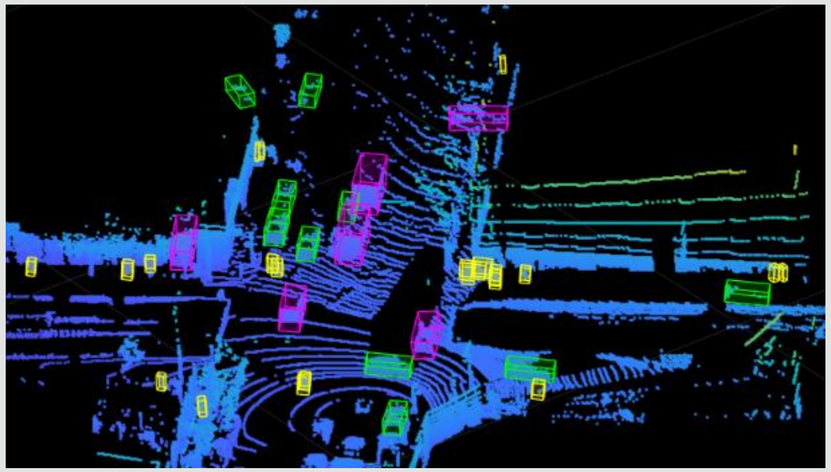
Bbox Validation

Bbox 크기 기준 필터링

대회 환경에서 탐지해야할 객체: ERP42, 드럼, 라바콘 등

➡ 모두 2.5m * 2.5m * 2.0m 이하

1. Clustering된 객체의 모든 Point들을 포함하는 최소 외곽 Bounding Box를 생성
2. 생성된 Bounding Box의 크기를 기반으로 2.5 * 2.5 * 2.0 이상 크기의 객체 제거



Kalman Filter Tracking

일시적인 관측 실패 또는 노이즈로인한 Centroid 위치 변화에도 객체 위치를 안정적으로 유지

Predict

이전 프레임에서 추적된 객체들을 등속운동 모델로 다음 위치 예측

x' = x + vx*dt

y' = v + vy*dt

Match (Greedy)

예측된 위치와 현재 프레임의 관측 클러스터 중심점(Centroid)간 가장 가까운 쌍부터 순서대로 할당

매칭 거리가 1.5m 이상이면 쌍과 관련없이 매칭 생략

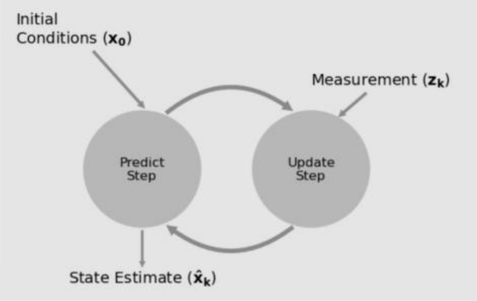
Update

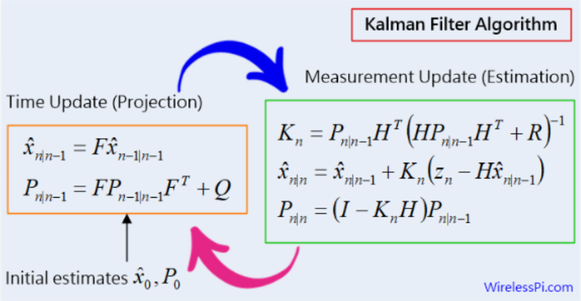
매칭이 성공된 쌍은 현재 프레임 측정값을 기반으로 위치 보정 → miss=0

매칭 실패 시, 예측값 유지 → miss+=1

신규 관측 → 새 트래커 생성

연속으로 5프레임 놓치면 트래커 삭제(miss > 5)





코드 구성

파라미터

Preprocess(ROI, Voxel Grid)

파라미터	설명
ROI_X_MIN/MAX	X축(전방) 관심범위 [m]
ROI_Y_MIN/MAX	Y축(좌우측) 관심범위 [m]
ROI_Z_MIN/MAX	Z축(후방) 관심범위 [m]
VOXEL_SIZE	Voxel Grid 격자 한 변 크기 [m]

DROR

파라미터	설명
DROR_MIN_NEIGHBORS	이웃 점 최소 개수 [개]
DROR_MIN_RADIUS	탐색 반경 최솟값 [m]
DROR_RADIUS_SCALE	거리 비례 반경 증가율
DROR_MAX_RADIUS	탐색 반경 최댓값 [m]

Ground Removal

파라미터	설명
GRID_CELL_SIZE	Grid 셀 크기 [m]
GRID_MAX_HEIGHT_DIFF	지면 판단 높이 차 임계값 [m]
GRID_MIN_POINTS	지면 판단 최소 포인트 수 [개]
GROUND_THREDH	RANSAC 지면 평면 거리 임계값 [m]

Euclidean Clustering

파라미터	설명
EUCLIDEAN_BASE_DIST	클러스터 탐색 기본 반경 [m]
EUCLIDEAN_DIST_SCALE	거리 비례 반경 증가율
EUCLIDEAN_MIN_CLUSTER_SIZE	클러스터 최소 포인트 수 [개]
CLUSTER_MERGE_GAP	클러스터 병합 거리 임계값 [m]
MIN_LENGTH/WIDTH/HEIGHT	Bbox 최소 크기
MAX_LENGTH/WIDTH/HEIGHT	Bbox 최대 크기

Kalman Tracking

파라미터	설명
dt	프레임 간격
Q	프로세스 노이즈(클수록 측정값 신뢰)
R	측정 노이즈 (작을수록 측정값 신뢰)
MATCH_DIST	매칭 허용 거리 임계값 [m]
TRACKER_MAX_MISS	트래커 삭제 전 최대 미 매칭 횟수

실습

LiDAR Sensor Data 확인

시뮬레이터 실행

```
source /opt/ros/kilted/setup.bash
cd ~/Autonomous-Racing-Simulator/simulate_ws
colcon build --symlink-install
gz sim -r ~/Autonomous-Racing-Simulator/simulate_ws/src/server/map/racemap.sdf
```

Terminal 1

ERP-42 생성

```
cd ~/Autonomous-Racing-Simulator/simulate_ws
source install/setup.bash
ros2 launch server spawn_car.launch.py
```

Terminal 2

Topic 확인

```
ros2 topic list
ros2 topic info /car1/scan/points
```

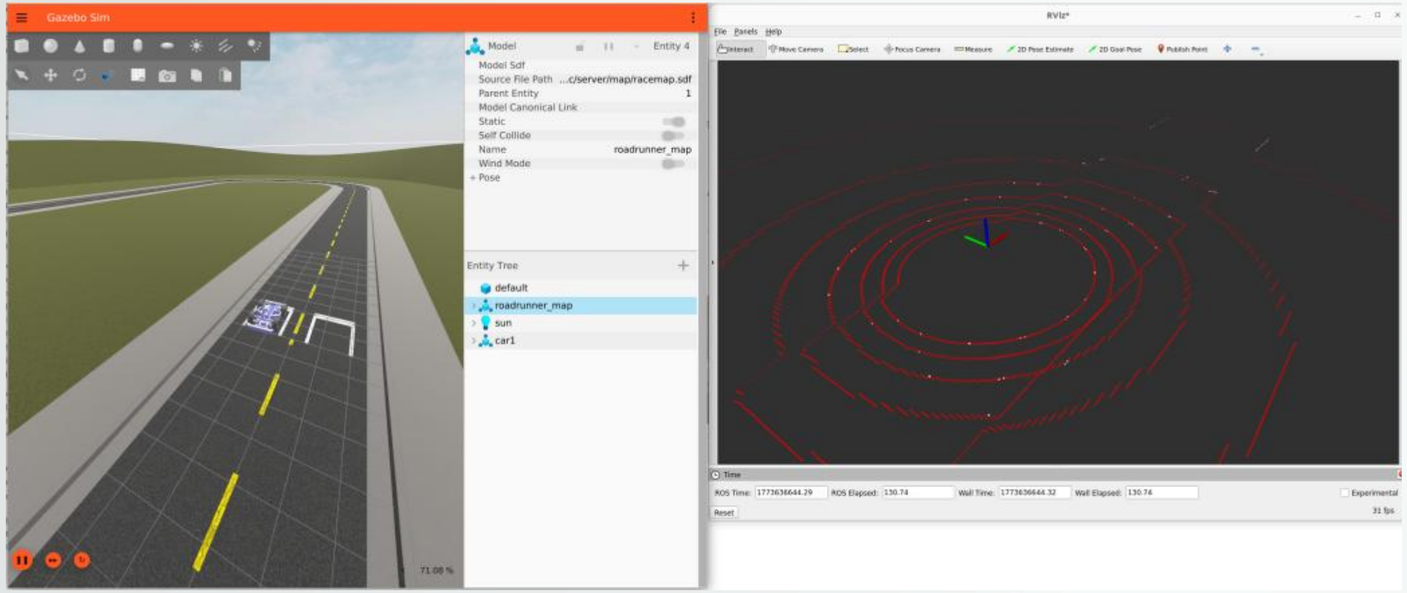
데이터 형태 확인

```
ros2 topic echo /car1/scan/points
```

Rviz2 이용해서 확인하기

```
rviz2
Fixed Frame - car1/lidar_link
add - by topic - /car1/scan/points
```

Terminal 3



실습

코드 실행 및 구조 확인

코드 실행

Terminal 3

Github Repository Clone

```
git clone https://github.com/Clothoid-R/Autonomous-Racing-Seminar.git
```

Build & Sourcing

```
cd ~/Autonomous-Racing-Seminar/lidar_ws
colcon build --packages-select lidar_detection
source install/setup.bash
```

실행

```
ros2 launch lidar_detection lidar_tutorial.launch.py
```

Terminal 4

시각화

rviz2
Fixed Frame - car1/lidar_link
add - by topic - /car1/scan/points
add - by topic - /preprocessed_points
add - by topic - /bounding_boxes

코드 구조

Import Statements - 라이브러리 불러오기
- rclpy(ROS2), Numpy, SciPy, message type 등 라이브러리 임포트

Parameter Configuration - 파라미터 설정
- 주요 파라미터 값 정의

Processing Function - 처리 함수
- roi_filter(): ROI 필터링
- voxel_downsample(): 다운샘플링
- remove_ground(): 지면 제거
- euclidean_clustering(): 클러스터링
- compute_bounding_box(): 바운딩 박스 계산
- is_valid_bbox(): 박스 유효성 검사

ROS2 Node Class - 노드 정의
- __init__(): 노드 초기화, 구독자/발행자 설정
- callback(): 메시지 수신 시 파이프라인 실행

Publishing Function - Rviz 시각화용 함수
- _publish_cloud(): 전처리 포인트 클라우드 발행
- _publish_boxes(): 바운딩 박스 마커 발행
- _make_box_marker(): 마커 생성 헬퍼

Entry Point - 진입점
- main(): ROS 노드 실행

실습

코드 및 파라미터 수정 실습

ROI
ROI_X_MIN, ROI_X_MAX # 전방 거리 (m)
ROI_Y_MIN, ROI_Y_MAX # 좌우 너비 (m)
ROI_Z_MIN, ROI_Z_MAX # 높이 범위 (m)

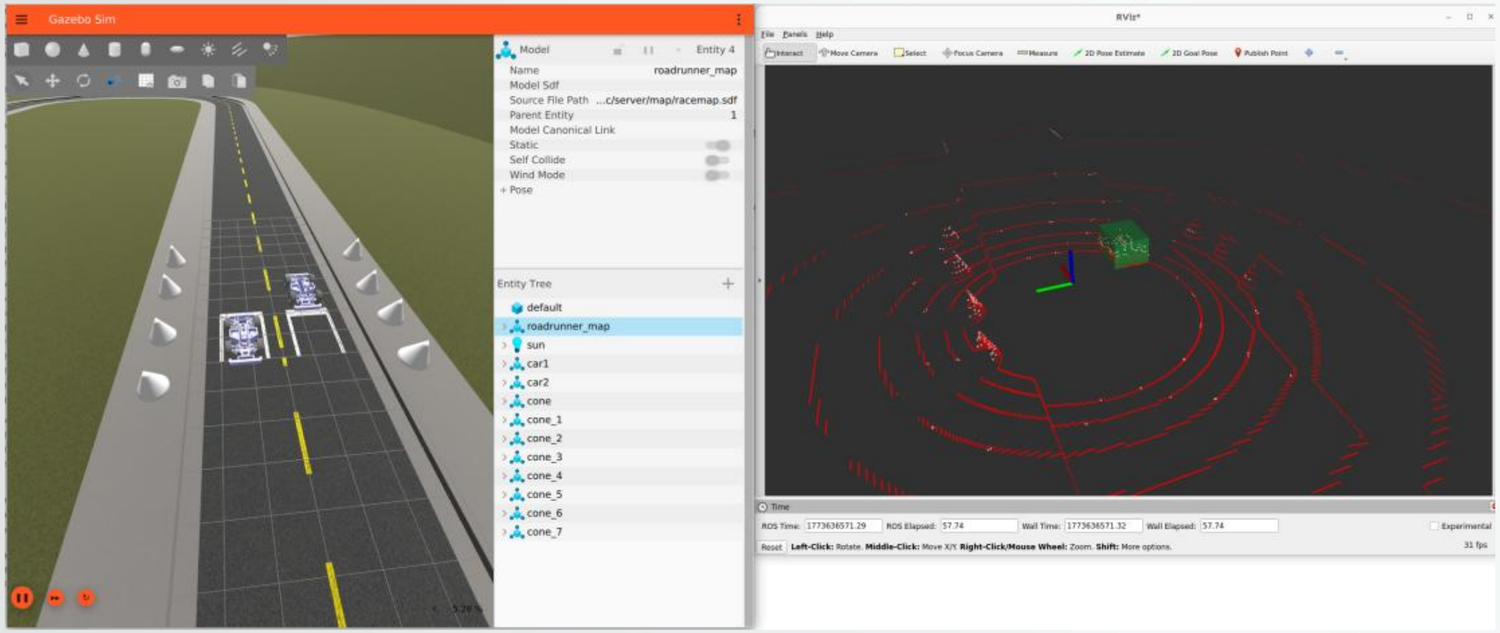
Voxel Downsampling
VOXEL_SIZE # 클수록 포인트 감소, 연산 속도 향상

Ground Removal
GRID_CELL_SIZE # 그리드 셀 크기 (m)
GROUND_DIFF # 지면 판단 높이 차이 (m)

Euclidean Clustering
CLUSTER_RADIUS # 클러스터 반경 (m)
MIN_CLUSTER_PTS # 클러스터 최소 포인트 수
MAX_CLUSTER_PTS # 클러스터 최대 포인트 수

Bounding Box
BB_MIN_LWH # 최소 크기 (m)
BB_MAX_LWH # 최대 크기 (m)

요구사항
ERP-42만 클러스터링 - 이외에 다른 객체 클러스터링 X
최종 Bbox 토픽 출력 약 20ms 이하

파라미터 (parameters) - line 30~60
처리 함수 - line 60~200
콜백 (callback) - line 225~250

} 이것만 수정